LOCAL RESEARCH / AUDITABLE BACKTESTING

QuantLab

美股量化研究系统

项目、使用与维护手册

中文版

从策略想法到可审计的历史实验：数据、因子、策略、成交、账户、指标、报告与桌面界面。

版本：项目 2026.10.22　核对日期：2026 年 10 月 22 日

适用目录：`/Users/yusa/ChatGPTWorkSpace/QuantLab`

本手册根据当前本地代码编写。它区分已实现功能、独立研究脚本和未来扩展；不将历史回测表现作为未来收益承诺。

目录

1 项目是什么，以及适合谁 **2**

1.1 能够完成什么 2

1.2 明确的能力边界 2

1.3 建议的阅读顺序 2

2 架构与目录导航 **3**

2.1 一条回测如何流经系统 3

2.2 顶层文件夹 3

2.3 核心模块与入口文件 4

3 安装与首次运行 **5**

3.1 准备 Python 环境 5

3.2 第一个离线示例 5

3.3 使用已有真实历史数据 6

4 桌面 App 操作指南 **6**

4.1 打开与连接 6

4.2 从选择文件到运行实验 6

4.3 编辑、结果与保存的区别 7

4.4 开发预览与重新打包 7

5 如何把策略想法变成可执行实验 **7**

5.1 先明确规则，再写代码 8

5.2 自己写策略的接口 8

5.3 可以读取什么 9

5.4 接入配置与运行 9

6 因子库与已有策略 **9**

6.1 因子与策略不是同一件事 10

6.2 当前因子的实际接口 10

6.3 已有策略的区别 11

7 数据、配置与执行模型 **11**

7.1 数据合同与复权 11
7.2 主要配置项 12
7.3 日内事件顺序与周频的含义 12
7.4 两种仓位转换方式 13
7.5 成本与缺失数据 13

8 如何阅读结果而不误读指标 13

9 归档、复现与独立审计 14
9.1 每次运行保存什么 14
9.2 重跑与迁移 15
9.3 现有策略的独立检查 15

10 现有研究专题如何使用 15

11 维护、测试与交付 16
11.1 推荐的修改流程 16
11.2 日常检查命令 16
11.3 依赖、备份与发布边界 16
11.4 手册本身的维护 17

12 故障排查与操作检查表 17

A 代码依据与版本说明 18

1 项目是什么，以及适合谁

QuantLab 是一个在本机运行的美股日线研究与回测系统。用户提供价格数据、策略代码和运行配置，系统按明确的时间顺序模拟历史交易，维护现金与持仓，计算绩效，并保存可以复核的完整实验档案。它的重点是正确性、可审计性和可复现性，而不是自动发现收益最高的策略。

项目包含三个层次：Python 回测引擎是计算核心；命令行与 Tauri 桌面 App 是两种操作入口；`researchProject/` 保存专题研究及其独立计算材料。桌面界面不会替换引擎的交易规则，研究专题也不等同于可以在策略菜单中直接运行的模块。

1.1 能够完成什么

- 读取本地 CSV/Parquet 日线，检查价格、股票代码、交易日和基准数据。
- 执行用户编写的 Python 策略，观察仅截至决策时点的历史，并在下一交易日开盘撮合。
- 模拟多头、美元账户、整数股、手续费和滑点，记录订单、成交、持仓与权益。
- 输出净值、收益、风险、成本、换手与基准比较，以及可离线打开的 HTML 报告。
- 冻结配置、策略、因子、实际输入数据及引擎代码，支持事后复算。
- 在桌面 App 中选择、编辑、保存策略与因子，设置参数、校验、运行或取消回测，浏览历史结果。

1.2 明确的能力边界

系统没有券商连接，不会下实盘订单；不支持做空、杠杆、分钟线、盘口撮合、限价单或退市清算。股票池是固定列表，不是历史时点成分数据库。核心运行与测试可离线完成，但安装依赖、可选 Yahoo 数据下载需要网络或事先准备的缓存。

桌面 App 当前依赖工作目录中的 `.venv/bin/python`，不内置完整 Python 运行时。把 App 单独复制到一台新电脑不等于完成安装。代码校验与只读数据接口用于约束可信策略，不是运行不可信 Python 的安全沙箱。

1.3 建议的阅读顺序

第一次使用可先阅读第 3、4 节；写策略看第 5、6 节；解释回测结果看第 7、8 节；维护代码、迁移环境与重现旧结果看第 9 至 12 节。所有终端示例默认在项目根目录运行，并已激活其虚拟环境。

2 架构与目录导航

2.1 一条回测如何流经系统



策略负责回答“希望持有什么”，引擎负责回答“何时、以什么价格、买卖多少、收取多少费用”。因子仅从已有历史中计算数值。将这些职责分开，才能区分投资规则变化与成交模型变化。

2.2 顶层文件夹

下表包含当前目录中的业务目录及常见生成目录。带点的目录可能被 Finder 隐藏。

目录	用途与维护方式
`src/quantlab/`	可安装的 Python 核心。引擎、数据、指标、报告与入口代码均在此。
`strategies/`	用户策略与惰性模板；直接决定交易逻辑，修改应保留规则说明与版本。
`factors/`	用户批准的纯计算因子；需要配置显式绑定，不是自动扫描执行的信号库。
`configs/`	YAML 实验定义；`examples/` 放基础示例。复制新配置可保留旧实验条件。
`data/raw/`	本地原始/冻结输入，包括合成样例与真实历史 Parquet。勿混淆数据来源。
`data/processed/`, `data/cache/`, `data/metadata/`	数据整理、缓存及来源信息的组织位置；是否读写以配置和具体脚本为准，并非所有目录都参与每次运行。
`runs/`	每次回测的独立归档。用于结果查看与复现，不应作为临时缓存随意删除。

目录	用途与维护方式
desktop/	React/TypeScript 界面及 Tauri 原生壳；内部 src/ 是前端，src-tauri/ 是 Rust。
scripts/	生成样例、校验数据、干净安装验证、独立算术审计等辅助脚本。
tests/	离线自动化测试，fixtures/ 保存可手算的测试数据。
researchProject/	独立研究专题；各自保存代码、数据、结果与报告，不能默认与核心引擎行为相同。
validation/	以前验收与研究运行的日志。它是历史证据，不代表今天已重新测试。
dist/, build/	App、wheel 等交付物及构建中间文件。源码改动后需要重新构建。
.quantlab-desktop/	桌面任务快照与日志；运行中不要清理。取消任务可能留下中间目录。
.venv/, .wheelhouse/	Python 环境与本地依赖安装包。虚拟环境通常不适合直接跨机器复制。
.git/	版本历史与远程配置；不是业务数据目录。
.pytest_cache/, .ruff_cache/	测试和工具缓存；可重建，不等于回测档案。

2.3 核心模块与入口文件

data/ 处理交易日历、输入校验与截断历史视图；strategy/ 定义合同、上下文、加载和检查；engine/ 管理时钟、撮合、订单与账本；costs/ 计算费用和滑点；metrics/ 定义统计口径；report/ 生成图表与 HTML；utils/ 提供序列化、路径与日志工具。这些子目录均位于 src/quantlab/。

config.py 解析严格配置，cli.py 提供命令行，archive.py 生成实验档案，desktop_bridge.py 为 UI 提供文件、配置、任务与结果操作。types.py 定义公共类型/异常，__init__.py 等是包入口。

根目录的 pyproject.toml 定义包和依赖范围；requirements.lock 固定已验收依赖；README.md 提供快速说明；AGENTS.md 记录项目与策略开发约束；V4 工作文档是原始规格。阶段验收文件和 DESKTOP_PROGRESS.md 是历史记录；出现与当前代码不一致之处，应先核对代码、测试与具体运行档案。

3 安装与首次运行

3.1 准备 Python 环境

项目声明 Python 3.12 或更新版本。已有环境可先检查，不必重复创建：

```
cd /Users/yusa/ChatGPTWorkSpace/QuantLab
source .venv/bin/activate
python --version
python -m pip check
quantlab --help
```

新环境的标准流程如下；应在确定使用哪个 Python 解释器后执行：

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.lock
python -m pip install -e '.[dev]'
python -m pytest -q
```

可编辑安装使 `src/` 的修改在该环境中直接生效。依赖固定文件不是全平台安装包；离线安装还需要对应平台的 wheel。当前虚拟环境可能引用外部 Python runtime，迁移或删除该 runtime 后应重建环境，而非只修改激活脚本。

3.2 第一个离线示例

```
python scripts/make_sample_data.py
python scripts/validate_data.py configs/default.yaml
quantlab strategies validate \
  strategies/fixed_weight_buy_and_hold.py \
  --config configs/examples/fixed_weight_buy_and_hold.yaml
quantlab backtest run \
  --config configs/examples/fixed_weight_buy_and_hold.yaml
```

生成器使用固定种子构造模拟价格；即使股票代码叫 AAPL、TSLA 或 SPY，也不是其真实历史行情。该例用于验证安装与链路。运行后终端输出归档路径与最终资产；用打印出的实际运行编号替换下面的 `RUN_ID`：

```
quantlab report open RUN_ID
```

在别的目录打开报告时传入 `--runs-dir/absolute/path/to/runs`，也可直接在浏览器打开归档内的 `report.html`。

3.3 使用已有真实历史数据

GitHub 发布不携带真实行情。新克隆应先生成合成 sample，并在 App 选择 sample 基础配置；以下历史实验需要先准备配置指向的数据文件。

现有周频研究配置指向本地 Yahoo 历史 Parquet，不会因为执行回测而重新下载。先校验再运行：

```
python scripts/validate_data.py \
  configs/weekly_equal_weight_aapl_tsla_2020_2025.yaml
quantlab strategies validate \
  strategies/weekly_equal_weight_aapl_tsla.py \
  --config configs/weekly_equal_weight_aapl_tsla_2020_2025.yaml
quantlab backtest run \
  --config configs/weekly_equal_weight_aapl_tsla_2020_2025.yaml
```

这个配置从 2020-01-03 的现金基线开始，以便在 2020-01-06 开盘执行首次指令。把开始日期直接改为首次买入日，会改变可用历史、第一笔交易与收益统计基线。

4 桌面 App 操作指南

4.1 打开与连接

在 Finder 中打开 `dist/QuantLab.app`。若未自动连接，进入“工作区设置”，填写项目根目录的绝对路径。它应包含 `pyproject.toml`、`src/quantlab/`、策略、配置以及可运行的 `.venv/bin/python`。原生 App 无需启动浏览器开发服务器。

左侧导航分为研究工作台、回测记录、数据目录和工作区设置。深色/浅色切换只影响显示。数据目录主要列出现有本地文件，不提供完整的数据采购、在线更新或清洗管理平台。

4.2 从选择文件到运行实验

1. 先选择接近目标实验的配置预设，检查策略文件、股票池、基准和日期。
2. 策略下拉菜单读取 `strategies/` 下的 Python 文件。选择后阅读实际代码；切换策略后重新检查参数兼容性。
3. 因子下拉菜单用于打开编辑。策略需要导入它时还必须绑定依赖，并在代码中调用对应函数；仅打开因子不会自动生成交易。
4. 在代码框编辑，点击保存或按 Command+S。多个文件可作为标签保留。新建策略复制项目模板，新建因子只有待实现的空模板。
5. 检查日期、本金、股票池、基准、本地行情路径；展开高级设置查看 JSON 参数、仓位模式和交易成本。
6. 点击校验策略；校验通过后运行回测。运行会保存修改、验证配置并冻结本次输入源码。查看日志以了解进度，必要时取消。
7. 确认结果区的运行编号，再查看净值、指标、成交明细和完整报告。

4.3 编辑、结果与保存的区别

保存代码会更新本地 Python 文件，但修改表单不自动覆盖原 YAML；本次参数保存在运行快照中。若希望作为长期配置复用，应另建/编辑 `configs/` 中的 YAML。

看到历史曲线不代表当前编辑已被测试。只有一次新的成功运行才对应新的配置与代码。历史列表最多返回最近 100 个条目，成交表最多显示前 2,000 笔；更早记录和完整成交仍在磁盘归档。单条损坏归档会单独显示失败，不阻断其他记录；失败标记优先于成功汇总，且成功记录必须有 complete 元数据。

文件在其他编辑器中被更改后，App 用内容指纹阻止静默覆盖。先复制需保留的草稿，再点击重新读取磁盘文件并确认放弃当前编辑。取消任务不会把未完成输出标记为有效结果；检查运行状态，不把残留目录当作成功回测。

4.4 开发预览与重新打包

日常使用已构建 App 不需要安装 Node/Rust；开发或重新打包需要 Node.js（Vite 6 支持的版本）、npm、Rust/Cargo 和 macOS 开发工具。本版本将 Vite 更新到兼容的 6.4.5 修复线、source-map-js 更新到 1.2.2；发布时 npm 审计返回零项已知漏洞。

```
cd desktop
npm ci
npm run desktop
# Build the production macOS app:
npm run bundle
```

开发浏览器预览可使用：

```
QUANTLAB_WORKSPACE=/Users/yusa/ChatGPTWorkSpace/QuantLab npm run dev
```

该入口仅监听 `127.0.0.1:1420`；原生版使用 Tauri IPC 调用 Python。新 App 在 `desktop/src-tauri/target/release/bundle/macos/QuantLab.app`。确认测试通过、退出旧 App 后再替换交付目录中的版本。修改源码不会自动更新已经打包的 App。

5 如何把策略想法变成可执行实验

5.1 先明确规则，再写代码

口头提出策略时，应提供以下内容：交易股票与观察基准；准确数学规则与阈值；首次建仓方式；信号计算区间；决策和执行时点；节假日处理；仓位与现金分配；手续费与滑点；开始/结束日期；缺失数据处理；允许的参数及不希望自动优化的限制。

例如“每周一等权”仍需明确是开盘成交，还是周一收盘决定次日成交。当前用户批准的 AAPL/TSLA 策略选择前一交易周最后一个交易日收盘生成指令，下一周首个交易日开盘成交。

交付通常包含 `strategies/name.py`、`configs/name.yaml`，若有新的数值特征再增加 `factors/name.py`。需要改变引擎时必须单独识别并测试，不能为改善收益暗中改变规则。表达清楚策略后，AI 也应遵循同一过程：写规则说明、实现、校验、回测、核查和报告。

5.2 自己写策略的接口

从 `strategies/_template.py` 复制起步，或在 App 中新建。下面是没有交易逻辑的最小骨架，不是额外推荐的研究策略：

```
from quantlab.strategy.base import BaseStrategy

STRATEGY_META = {
    "name": "my_strategy", "version": "0.1.0",
    "description": "Document the exact approved rule",
}

class Strategy(BaseStrategy):
    def warmup_bars(self):
        return 0

    def generate_target_weights(self, dt, data, ctx):
        return None

    def decision_metadata(self):
        return {}

def build_strategy(params):
    return Strategy(params)
```

`build_strategy(params)` 创建实例；基类复制参数，策略可保存其自身的历史状态。`decision_metadata()` 用于返回可审计的决策解释，例如因子数值和触发原因，不应另行下单。

返回值语义必须区分：`None` 表示没有新指令；`{}` 表示目标全现金；字典表示整个目标组合，没有列出的现有持仓目标为零。权重必须有限、非负、属于可交易股票池，且不超过单股/总敞口限制。若权重之和小于 1，剩余部分对应现金目标。价格漂移后不会自动强制恢复权重。

5.3 可以读取什么

接口	语义
`data.history(symbol,bars=None,end=None)`	只返回决策时点之前可见的历史副本，可限制条数和结束时间。
`data.current(symbol)`	当前可见 bar 或 `None`，应处理缺失。
`data.as_of`	数据视图的截止交易日。
`ctx.cash, ctx.equity`	当前只读现金和权益。
`ctx.positions, ctx.universe`	只读股数映射与允许交易的股票池。
`ctx.next_session_date`	已知交易日历的下一 session 日期，不是未来价格。
`ctx.seed, ctx.execution_sizing`	随机种子及实际仓位转换模式。

基准历史可用于观察，但不能因此把基准当作可交易股票。`warmup_bars()` 只累计回测区间内的可观测 bar；没有自动加载区间外训练历史，需在配置中预留所需数据。

策略不应下载行情、读取未来数据、调用引擎私有对象或直接修改账本。校验器会检查常见违规导入和文件 I/O，但不能证明任意 Python 程序安全，也不能证明投资逻辑有效。

5.4 接入配置与运行

复制现有 YAML，修改 `strategy.path`、`params`、股票池和数据路径；相对路径以 YAML 所在目录为基准。CLI 的显式 `--strategy` 路径则相对于当前终端目录，并覆盖 YAML 指定策略。

```
quantlab strategies validate strategies/my_strategy.py \
  --config configs/my_strategy.yaml
quantlab backtest run --config configs/my_strategy.yaml
```

校验通过只意味着基本合同满足。仍需检查极端输入、关键交易日、仓位与现金，以及输出是否真正符合书面规则。

6 因子库与已有策略

6.1 因子与策略不是同一件事

因子是计算出来的特征；策略是使用特征、时间和账户状态决定交易的规则。例如每日 AAPL 相对 SPY 的收益差为

$$f_t = \left(\frac{P_t^A}{P_{t-1}^A} - 1\right) - \left(\frac{P_t^M}{P_{t-1}^M} - 1\right).$$

这里使用系统提供的复权价格。该差值不是 beta 调整后的残差，也不是对个股“高估/低估”的直接证明。以 ±0.01 为阈值是相差正负 1 个百分点，不是相对收益差再乘百分比。

因子不决定买入多少，也不决定何时交易。同一个因子可以被不同策略使用；反过来，“每周恢复 50/50”只是固定权重再平衡，不需要收益预测因子，所以不会自然产生因子库文件。

6.2 当前因子的实际接口

`factors/return_spread.py` 提供 `daily_return_spread` 和 `weekly_return_spread`。前者使用同步的两日收盘，后者使用两个相邻完整交易周的最后收盘。周收益不能用最后一天的单日收益替代，也不简单等于一周日收益的算术和。

周函数应在策略已确认本周最后一个交易日的收盘时调用，否则可能把未完成周当作完整周。历史不足返回 `None`；端点不同步、非正价格等会报错。函数返回起止日期、个股收益、基准收益和差值，用于审计。

绑定示例（位于 `configs/` 中）：

```
strategy:
  path: ../strategies/aapl_weekly_relative_reversal.py
  params:
    start_date: '2020-01-06'
    end_date: '2025-12-31'
    threshold: 0.01
  rebalance_schedule: daily
  factor_paths:
    factors.return_spread: ../factors/return_spread.py
execution:
  sizing: decision_relative
```

这是配置片段，其他部分应保留对应完整配置。策略中通过 `fromfactors.return_spreadimportweekly_return_spread` 调用。加载器按模块名显式绑定、检查并冻结源码，不自动执行因子目录内所有文件。

6.3 已有策略的区别

文件 / 实验	实际规则
`fixed_weight_buy_and_hold.py`	初次发出给定权重，随后不主动再平衡。基础示例各半 AAPL/TSLA。半仓 AAPL 对照实验复用此实现。
`weekly_equal_weight_aapl_tsla.py`	从 2020-01-06 开始，下周首个交易日开盘恢复各 50% 目标；无需因子。
`aapl_weekly_relative_reversal.py`	初始一半资金买 AAPL；之后周收益差低于 -1pp 买半数现金，高于 +1pp 卖半数股数，否则保持。
`_template.py`	惰性起点，返回 `None`，不代表有效投资规则。

相对反转策略首次建仓优先，不在同一时点叠加因子交易；现金不生息，买入预算含手续费，卖出向下取整。阈值相等（代码含 10^{-12} 的边界容差）不交易。它要求 `decision_relative` 模式，不能与普通目标权重转换随意互换。

7 数据、配置与执行模型

7.1 数据合同与复权

本地输入为 long format，每行一只股票一个交易日：

```
date,symbol,open,high,low,close,volume,adj_close
```

`date` 是无时区的交易日标签，系统决策与成交时间戳使用 UTC。数据需要覆盖股票池和基准，并符合 XNYS 交易日历。重复日期/股票、非法价格、负成交量、非交易日和缺少所需股票会明确失败。

在 `adjusted` 模式下，使用 $a_t = \text{adj_close}_t / \text{close}_t$ 同比缩放 OHLC；不再单独入账拆股和分红，成交量保持源值。整数股是复权研究单位，不应解释为历史原始股数。`raw` 模式不处理公司行动，显式非零分红或非平凡拆股标记会被拒绝；漏报行动仍会产生偏差。

更新数据时应保留旧文件，写入新文件并记录来源、下载日期、区间、复权政策和校验结果。复权历史可能被源端修订；固定旧输入才能解释两次结果差异。Yahoo adapter 是可选便利入口，不是唯一权威数据源。需要时先安装 `'.[online]'`，显式选择 `data.provider:yahoo`；桌面主流程仍以本地文件为主。

7.2 主要配置项

配置段	控制内容
`backtest`	起止日期、本金（正数）、USD 基准货币。
`data`	provider、path、1d 频率、复权方式、基准、来源描述和异常收益告警阈值。
`universe`	非空、无重复的可交易代码列表，以及固定列表/当前成分来源声明。
`strategy`	源码、参数、决策资格频率和因子模块到源码路径的映射。
`execution`	收盘决策、下一开盘执行、整数股及 sizing 模式。
`commission, slippage`	手续费模型、最低费和基点滑点。1 bps 等于 0.01%。
`portfolio`	禁止做空和负现金，单股权重与总敞口限制。
`analytics`	年化交易日数（默认 252）、无风险率、滚动窗口。
`report`	报告开关、是否自动打开、运行输出目录。
`reproducibility`	随机种子。仅固定种子不能替代数据和代码快照。

Pydantic 严格校验未知字段和非有限数值；YAML 重复键也会报错。配置路径相对于 YAML，而不是 App 当前界面位置。

7.3 日内事件顺序与周频的含义

1. 当日开盘处理上一交易日收盘形成的待执行目标。
2. 按当日开盘标记已有仓位，冻结目标转换所需权益。
3. 结合滑点和整数股计算订单，先卖后买，买入按代码字母顺序处理。
4. 现金不足时确定性缩量并取消余量，不允许借款；零成交不收最低手续费。
5. 当日收盘估值、生成截断历史视图、调用策略、记录日终状态。

`rebalance_schedule:weekly` 指“每周首个交易日收盘具有决策资格”，不等于“周一开盘成交”。已有周一策略使用 `daily` 每天检查下一 session 是否进入新周，实际只在周末边界发出目标。已知交易日历可以提前使用，未来价格不能。

7.4 两种仓位转换方式

普通 `target_weights` 按开盘权益和目标权重转换成整数数量。即使权重写为各半，费用、滑点、整数舍入与现金不足都可能使实际权重偏离 1:1。

`decision_relative` 在收盘把权重变化转换并冻结为现金/股数比例，下一开盘按该比例执行。这样周末跳空后，“用一半剩余现金买入”仍保持原意。此模式目前仅支持单只可交易股票，用于已有 AAPL 相对反转；不是多资产通用仓位算法。

7.5 成本与缺失数据

手续费支持零费、每股费加最低费、成交额基点费；滑点支持零或基点模式，买入加价、卖出减价。滑点已包含在成交价中，不能再重复扣现金。买入手续费进入平均成本，卖出净收入计入已实现损益。

缺少执行 bar、开盘价无效或成交量为零时取消订单；不得借未来价格补成交。持仓缺少当日 bar 时用最后已知价格估值并告警；缺少基准 bar 则失败。最终一日的待执行指令取消，但期末仓位不会强制卖出。

日线最终成交量只用于零量可交易标记；真实开盘时尚不知道当日总量，所以这是模型假设，不是盘中流动性模拟。账本持续检查

$$E_t = C_t + MV_t = E_0 + \text{RealizedPnL}_t + \text{UnrealizedPnL}_t.$$

8 如何阅读结果而不误读指标

先核对数据来源、开始/结束日期、首次买入日、基准、成本和警告，再看收益。较低回撤可能来自较低股票敞口，而非更好的选时；相对反转应与匹配的初始半仓买入持有对照比较，也可同时观察 SPY。

令 E_t 为每日收盘权益，$r_t = E_t/E_{t-1} - 1$。首个 session 收盘为现金基线，日收益统计排除第一行占位零收益。核心口径为：

$$\begin{aligned}
R_{\text{total}} &= E_T/E_0 - 1, \\
\text{CAGR} &= (E_T/E_0)^{1/Y} - 1, \quad Y = \frac{d_T - d_0}{365.25}, \\
\sigma_{\text{ann}} &= s(r_t)\sqrt{252}, \\
\text{Sharpe} &= \frac{\overline{r_t - r_f}}{s(r_t - r_f)}\sqrt{252}, \\
\text{MDD} &= \min_t \left(\frac{E_t}{\max_{u \le t} E_u} - 1 \right).
\end{aligned}$$

252 可由配置改变；s 使用样本标准差，年化无风险率按几何方式转成日率。Sortino 分母为所有观测的负超额收益平方均值的平方根；Calmar 为 CAGR 除以最大回撤绝对值。回撤持续时间按日历日计算，未修复回撤截至最后观察日。

换手为双边实际成交额除以前一收盘权益，再按收益区间数年化，不除以二。平均现金与敞口包含初始现金日。Beta、alpha、tracking error 和 information ratio 使用与策略对齐的基准日收益；基准是首日收盘开始、无费用且可分数股的买入持有曲线，不等同于策略的真实成交账户。

无法定义的比率为 `null`/N/A，不应替换为零；成交笔数不是已完成往返交易次数。系统不为买入持有强行给出往返胜率。HTML 报告内嵌图片，可离线阅读；数值审计应以 CSV/Parquet/JSON 原始输出为依据。

9 归档、复现与独立审计

9.1 每次运行保存什么

运行目录名包含 UTC 时间、策略名和摘要，重复运行不会覆盖旧目录。下列文件用于回答“当时到底运行了什么”：

文件	作用
`config.snapshot.yaml`	指向冻结输入的解析配置。
`config.original.json`	初始解析配置，用于追溯原路径。
`strategy.snapshot.py`, `factors.snapshot/`	策略与显式依赖因子源码。
`data.snapshot.parquet`	当次使用的调整前输入 panel，避免重新联网。
`code.snapshot/`, `requirements.snapshot.txt`	当时的核心代码、安装描述和依赖版本。
`metadata.json`, `manifest.json`	环境、版本、Git 状态、指纹与文件 SHA-256。清单不含自身及持续写入的日志。
`equity.csv`, `returns.csv`, `benchmark.csv`	权益、日收益和基准。
`decisions.csv`, `orders.csv`, `fills.csv`	决策原因、请求/取消/成交及成本。
`positions.parquet`	逐日持仓。
`summary.json`, `metrics.json`, `warnings.json`	汇总、完整指标与告警。
`report.html`, `logs.txt`, `reproduce.sh`	人类可读报告、运行日志和复现入口。
`failure.json`	失败标记；不能把该目录当作成功实验。

报告关闭时可以没有 HTML。源码有未提交修改时，Git commit 本身不足以确定运行内容；冻结源码和哈希更直接。不要只备份截图或 PDF 而丢失输入和账本。

9.2 重跑与迁移

同一环境中运行：

```
sh runs/RUN_ID/reproduce.sh
```

这会生成新的运行目录，不改写旧结果。跨环境应先创建新的虚拟环境，安装该次 `requirements.snapshot.txt` 与 `code.snapshot/`，再执行相应复现命令。归档移动后，配置中的绝对路径可能失效；保留原件，另存修改路径后的配置，并记录修改。不要用当前源码冒充历史源码复算。

同输入、同规则、同环境应得到一致经济结果；跨平台浮点差异不保证文件逐字节相同。对比时同时检查交易日期、股数、费用、每日权益和指标，而不是只看最终收益。

9.3 现有策略的独立检查

```
python scripts/audit_weekly_relative_reversal.py runs/RUN_ID
```

该脚本独立重算现有周频反转规则的信号、现金/股数与每日权益，不调用策略和引擎实现。它适用于对应规则和成本条件，不是任意策略的通用审计器；更改规则后需相应更新独立计算依据。

10 现有研究专题如何使用

`researchProject/RESEARCH_INDEX.md` 是研究入口。当前四个专题分别研究：

1. `01_aapl_tsla_rebalancing/`：AAPL/TSLA 目标权重和日历再平衡频率。
2. `02_threshold_partial_rebalancing/`：阈值触发与部分纠偏对交易频率、跟踪和表现的影响。
3. `03_time_and_asset_robustness/`：入场时间、持有期限和资产组合的稳健性。
4. `04_rebalancing_mechanisms/`：分散化增长与相对买入持有净超额收益的区别。

每个专题按自身 README/审计说明复算，阅读其报告、数据字典、协议、结果与代码，而不是直接在 App 选择报告文件。后续专题有独立计算引擎；不能因名称相近就假设与主引擎逐项相同。

尤其第 04 个专题区分无摩擦收盘到收盘恒等式、合成实验和原有开盘成交约定。数学分解项不能直接替代含成本的开盘回测收益。固定资产选择、重叠窗口和参数比较也不能自动构成样本外证据。要把研究发现转成正式策略，应重新明确规则、写入策略/因子合同、增加测试并验证主引擎结果。

11 维护、测试与交付

11.1 推荐的修改流程

1. 先写变更目的与预期行为，确认修改属于策略、数据、引擎还是界面。
2. 保存原配置与参考运行，使用版本控制记录源码；不要覆盖旧行情文件。
3. 实现最小必要改动。引擎改动必须增加能发现真实错误的测试，保留未来价格扰动等防前视检查。
4. 运行相关测试和完整套件；必要时做一条本地离线端到端回测。
5. 对有交易语义影响的改动，比较订单、成交、权益和成本，说明结果变化原因。
6. 更新手册/README 和配置说明；界面改动后重新构建并验证实际 App。

11.2 日常检查命令

```
python -m pytest -q
python -m pip check
# Desktop checks, from desktop/:
npm ci
npm run build
cargo check --manifest-path src-tauri/Cargo.toml
npm run bundle
```

本次文档编写于 2026-10-22 重新运行了 Python 完整测试：**136 项通过**。这不表示所有后续改动都会通过，也不替代 App 发布前的原生窗口检查。测试覆盖账本、费用、缺失数据、策略校验、防前视、配置、归档复现与桌面桥接。

`scripts/check_clean_install.py` 可在临时环境验证 wheel 安装、测试和离线样例；前提是 `dist/` 中已有项目 wheel，`.wheelhouse/` 中有全部所需平台依赖。没有这些准备时，不应把脚本失败误判为策略失败。

11.3 依赖、备份与发布边界

依赖升级应在新环境中做，核对 `pyproject.toml` 的兼容范围，更新相应锁文件并跑回归。前端使用 `package-lock.json`，Rust 使用 `Cargo.lock`；不要只升级一端而忽略原生接口兼容性。

建议备份源码、配置、原始数据、完整重要运行档案与研究协议。环境和构建缓存可重建，但若要求完全离线恢复，还需备份兼容的 wheel 及安装工具。清理前先确认没有运行中任务；不要把 `runs/` 与 `.quantlab-desktop/` 当成同一种目录。

2026-10-22 用户批准将现有简单策略和因子一并发布。2026.10.22 版本包含本体、桌面源码、配置、测试及本手册；不包含独立 `researchProject/`、真实行情、运行档案、环境与缓存。现有 GitHub 仓库在发布核对时为公开状态；本次保留其可见性。维护时核对发布清单与 Git 历史；`.gitignore` 不会移除已提交内容。发布边界见根目录 `PROJECT_SCOPE.md`。

11.4 手册本身的维护

本文件是自包含 LaTeX 源码，无外部图片依赖。中文使用 XeLaTeX 和系统宋体/黑体（或 Fandol 字体回退）；英文使用 TeX Gyre 字体。修改后编译两次以更新目录与交叉引用，查看 PDF 是否有溢出、缺字或断裂代码。用临时输出目录保存辅助文件，最终 cn/en 目录只保留源文件和 PDF。

12 故障排查与操作检查表

现象	先检查什么
命令不存在 / 包无法导入	当前目录与虚拟环境；`python--version`、可编辑安装、`quantlab--help`。
App 连接失败	工作区是否是项目根目录；`.venv/bin/python` 能否运行；目录是否移动、依赖是否完整。
配置不合法	未知/重复 YAML 字段、日期顺序、参数类型；相对路径基于 YAML。
策略校验失败	元信息、基类、构造函数、返回权重、违规导入、显式因子绑定和 sizing 模式。
缺少因子模块	`factor_paths` 的模块名是否与 import 一致；是否确实绑定该源码。
没有成交	是否返回 `None`、预热不足、日期未到、资金不足，或订单因无开盘/零成交量被取消。看 decisions/orders。
周频晚一天	是否误把 weekly 决策资格当作周一开盘执行；检查下一交易日日期。
权重不是精确各半	整数股、成本、开盘跳空和现金约束；区分目标和实际权重。
曲线没有变化	当前显示是否为历史运行；修改代码/表单后是否产生新的成功编号。
保存发生冲突	外部文件已更改；保留草稿，再重载并人工合并，勿强行覆盖。
运行失败 / 报告为空	查 `failure.json`、日志、报告开关；不要仅凭目录存在判断成功。
复现结果不同	数据/代码/参数/依赖/种子是否一致；是否误用新引擎；归档路径是否失效。
更新源码但 App 不变	是否重新运行 bundle，并打开新生成的 App 而非旧副本。

每次研究提交前核对：规则是否事先确定；数据是否真实且来源可追溯；时点是否无前视；成本是否明确；异常是否披露；是否有合适对照；是否保存完整运行档案。收益不佳应如实报告，不能自动修改阈值直到回测好看。

A 代码依据与版本说明

本手册于 2026-10-22 核对以下本地材料：`README.md`、`AGENTS.md`、`pyproject.toml`、`configs/`；核心 `config.py`、`cli.py`、策略基类/上下文、因子函数、绩效实现、桌面桥接；`desktop/README.md`；`researchProject/RESEARCH_INDEX.md` 与辅助脚本。本次本体版本标识为 `2026.10.22`，对应 Git 标签 `v2026.10.22`；具体提交由仓库标签解析。审查范围和证据见 `AUDIT_2026.10.22.md`。

本文以实际实现为准。旧 README 的 V2 路线图和“唯一官方示例”描述对应较早阶段；当前已经存在用户批准的追加策略、桌面界面及研究专题。手册不会据此推断尚未实现的实时交易、通用自动寻优、完整因子评估平台或跨机器独立安装能力。